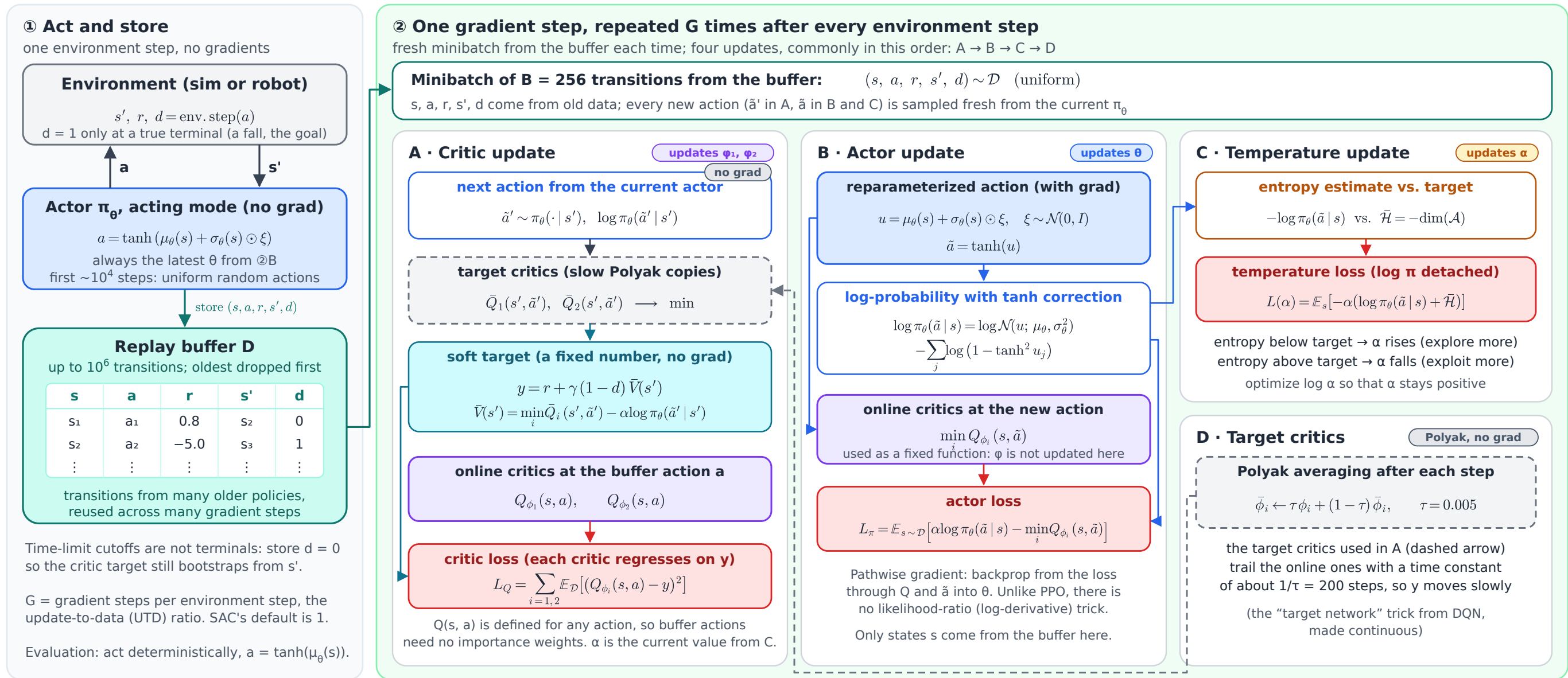


SAC cheat sheet · robotics / continuous control

Soft Actor-Critic as used today (SAC v2: twin Q critics, target critics, automatic temperature). Off-policy: every environment step goes into a replay buffer, and every step also triggers gradient updates.

 trainable network  target network (Polyak copy)  arithmetic, no grad  loss term  replay buffer  updates ϕ_1, ϕ_2  updates θ  updates α which parameters a loss updates



Pseudocode

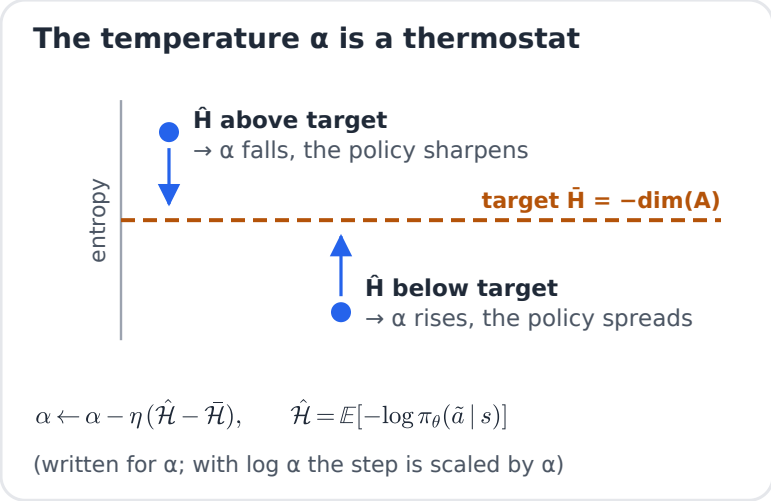
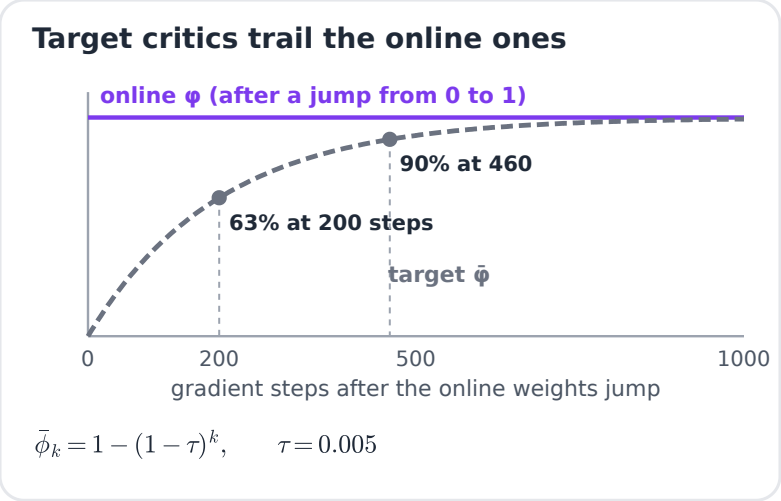
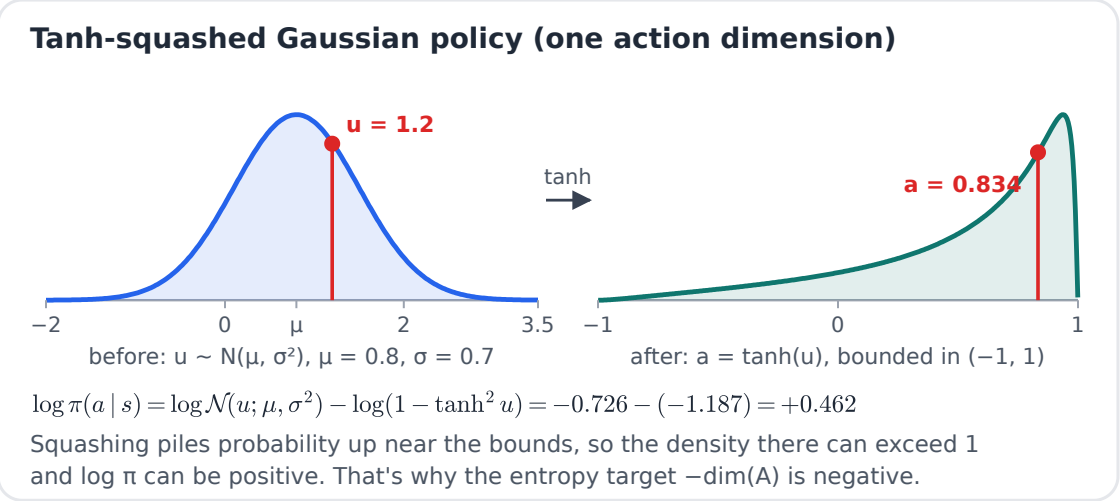
```
init  $\pi_\theta, Q_{\phi_1}, Q_{\phi_2}, \text{targets } \bar{\phi}_i \leftarrow \phi_i, \log \alpha$ , empty buffer D
for each environment step:
   $a \sim \pi_\theta(\cdot | s)$  (uniform random during warm-up)
   $s', r, d = \text{env.step}(a)$ ; D.add(s, a, r, s', d)
  for g in 1..G: # G = UTD ratio
     $(s, a, r, s', d) \sim \mathcal{D}$  # minibatch of B
     $\tilde{a}' \sim \pi_\theta(\cdot | s')$  # no grad
     $y = r + \gamma(1-d)(\min_i \bar{Q}_i(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}' | s'))$ 
    A: step  $\phi_1, \phi_2$  on  $\sum_i (Q_{\phi_i}(s, a) - y)^2$ 
     $\tilde{a} = \tanh(\mu_\theta(s) + \sigma_\theta(s) \cdot \xi)$  # reparameterized
    B: step  $\theta$  on  $\alpha \log \pi_\theta(\tilde{a} | s) - \min_i Q_{\phi_i}(s, \tilde{a})$ 
    C: step  $\log \alpha$  on  $-\alpha(\log \pi_\theta(\tilde{a} | s) + \bar{\mathcal{H}})$ 
    D:  $\bar{\phi}_i \leftarrow \tau \phi_i + (1 - \tau) \bar{\phi}_i$ 
  s ← reset() if the episode ended, else s'
```

Key facts

- Off-policy: each transition is reused across many updates; nothing is discarded after an update.
- Only s, a, r, s', d come from old data. Every new action is sampled from the current π_θ , so no importance weights are needed (unlike PPO's ratio ρ).
- Twin critics and a min fight Q overestimation (clipped double-Q, borrowed from TD3).
- The actor gradient flows through the critic (reparameterization), not through a likelihood ratio.
- α turns “how random should the policy be” into a target: keep entropy near $\bar{\mathcal{H}} = -\dim(\mathcal{A})$.
- Continuous densities can exceed 1, so $\log \pi$ can be positive and $\bar{\mathcal{H}}$ is negative. That's fine.
- Target critics move slowly ($\tau = 0.005$), which keeps the regression target y stable.
- Time-limit truncation is not a terminal: keep $d = 0$ and let y bootstrap from s' .
- High UTD (many gradient steps per env step) improves sample efficiency but can destabilize Q.

What each piece does, and how SAC differs from PPO

Plots are exact for the stated numbers. The SAC vs PPO table is for robotics and continuous control.



SAC vs PPO for robotics and continuous control

	PPO	SAC
Data use	On-policy: each rollout is used for K epochs, then discarded	Off-policy: replay buffer of up to 10^6 transitions, reused many times
Sample efficiency	Low; leans on cheap, massively parallel simulation	High; practical when samples are costly, e.g. on real robots
Policy	Gaussian, σ often state-independent; the env clips actions	Tanh-squashed Gaussian, state-dependent σ ; bounded by construction
Critic	$V(s)$, trained on GAE targets from fresh rollouts	Twin $Q(s, a)$ with slow target copies and a min over the two
Policy gradient	Likelihood ratio (REINFORCE-style), clipped by ϵ	Reparameterized: backpropagated through Q into the action
Exploration	Set by σ , plus an optional entropy bonus c_2	Entropy is part of the objective; α is tuned to a target entropy
Stability knobs	Clip ϵ , KL early stopping, number of epochs K	Target smoothing τ , clipped double-Q, α , the UTD ratio G
Update schedule	After each rollout: K epochs $\times$ minibatches	G gradient steps after every single environment step
Typical home	Massively parallel GPU simulators, e.g. legged locomotion	Sample-limited training, real robots, standard MuJoCo benchmarks
Action spaces	Discrete or continuous	Designed for continuous actions; discrete variants exist

Worked example: one gradient step's numbers ($\gamma = 0.99$, $\alpha = 0.2$, 6-D actions so $\bar{H} = -6$)

A · critic targets for two transitions

	r	d	$\min_i \bar{Q}_i(s', \tilde{a}')$	$\log \pi(\tilde{a}' \mid s')$	$\bar{V}(s')$	y	$Q_{\phi_1}(s, a)$	$Q_{\phi_2}(s, a)$
not terminal	1.0	0	9.8	-1.5	10.10	11.00	10.6	11.3
robot fell	-5.0	1	(unused)	(unused)	—	-5.00	-3.9	-4.4

$\bar{V}(s') = \min(10.2, 9.8) - 0.2 \times (-1.5) = 9.8 + 0.3 = 10.10$, and $y = 1.0 + 0.99 \times 10.10 = 11.00$.
Squared errors: $(10.6 - 11.0)^2 = 0.16$ and $(11.3 - 11.0)^2 = 0.09$. Both critics move toward the same y .
Terminal: $d = 1$ removes the bootstrap, so $y = r = -5.00$ whatever the target critics say.

B · actor loss and C · temperature at one state

$\log \pi_{\theta}(\tilde{a} \mid s)$	$\min_i Q_{\phi_i}(s, \tilde{a})$	L_{π}	$\hat{\mathcal{H}} - \bar{\mathcal{H}}$	α moves
-1.2	$\min(8.3, 8.0) = 8.0$	-8.24	$1.2 + 6 = 7.2$	down

$L_{\pi} = 0.2 \times (-1.2) - 8.0 = -8.24$. Minimizing it raises $Q(s, \tilde{a})$ and lowers $\log \pi$, i.e. keeps the policy spread out.
 $\partial L(\alpha) / \partial \alpha = \hat{H} - \bar{H} = 7.2 > 0$, so a gradient step lowers α .
Early in training entropy sits above the target and α decays.

Common pitfalls

- Time limits stored as $d = 1$
biases Q near the cutoff; store $d = 0$ and bootstrap
- Gradients leaking into the wrong loss
detach y and $\log \pi$ in $L(\alpha)$; L_{π} must not step ϕ
- Missing the tanh correction in $\log \pi$
entropy is mis-measured, so α chases the wrong target
- Evaluating with sampled actions
understates the policy; evaluate with $a = \tanh(\mu_{\phi}(s))$
- Env bounds other than $[-1, 1]$
rescale $a_{\text{env}} = c + h \odot a$ instead of clipping in the env
- Raising UTD without regularizing
 Q can diverge; ensembles, dropout or layer norm help

Formulas

SAC v2 with twin critics and automatic temperature. $\mathbb{E}$ over the buffer means averaging over the minibatch; $\tilde{a}$ ' and $\tilde{a}$ are always fresh samples from the current policy π_{θ} .

1 • Setup

transition	$(s, a, r, s', d) \in \mathcal{D}, \quad a \in \mathcal{A} \subseteq \mathbb{R}^{\dim \mathcal{A}}$
done flag	$d = 1$ only at a true terminal; time limits keep $d = 0$
networks	π_{θ} (actor), Q_{ϕ_1}, Q_{ϕ_2} (critics), $\bar{Q}_i = Q_{\bar{\phi}_i}$ (targets)
temperature	$\alpha = e^{\log \alpha} > 0$, learned
hyperparameters	$\gamma, \tau, \bar{\mathcal{H}}, B$ (batch), G (UTD), $ \mathcal{D} $ (buffer)

2 • Maximum-entropy objective

objective	$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [\gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)))]$
entropy	$\mathcal{H}(\pi(\cdot \mid s)) = \mathbb{E}_{a \sim \pi} [-\log \pi(a \mid s)]$
$\alpha \rightarrow 0$ recovers the standard RL objective. Larger α rewards staying random.	

3 • Soft Bellman equations and policy improvement

soft Q	$Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot \mid s, a)} [V(s')]$
soft V	$V(s) = \mathbb{E}_{a \sim \pi} [Q(s, a) - \alpha \log \pi(a \mid s)]$
improvement	$\pi_{\text{new}}(\cdot \mid s) = \arg \min_{\pi'} D_{\text{KL}} \left(\pi'(\cdot \mid s) \parallel \frac{\exp(Q(s, \cdot) / \alpha)}{Z(s)} \right)$
optimum	$\pi^*(a \mid s) \propto \exp(Q^*(s, a) / \alpha)$ (Boltzmann in Q)
The improvement step is an I-projection onto the policy class (π' sits in the first slot of the KL): the reverse direction of RL's Razor's M-projection view of the policy gradient step.	

4 • Critic update

next action	$\tilde{a}' \sim \pi_{\theta}(\cdot \mid s')$ (one fresh sample, no grad)
soft target value	$\bar{V}(s') = \min_{i=1,2} \bar{Q}_i(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}' \mid s')$
target	$y = r + \gamma(1 - d) \bar{V}(s')$
loss (each critic)	$L_Q(\phi_i) = \mathbb{E}_{\mathcal{D}} [(Q_{\phi_i}(s, a) - y)^2]$ (often $\times \frac{1}{2}$)
gradient	$\nabla_{\phi_i} L_Q = 2 \mathbb{E}_{\mathcal{D}} [(Q_{\phi_i}(s, a) - y) \nabla_{\phi_i} Q_{\phi_i}(s, a)]$
y is treated as a constant: no gradient flows into the targets or the actor through y.	

5 • Actor update

reparameterize	$u = \mu_{\theta}(s) + \sigma_{\theta}(s) \odot \xi, \quad \xi \sim \mathcal{N}(0, I), \quad \tilde{a} = \tanh(u)$
log-probability	$\log \pi_{\theta}(\tilde{a} \mid s) = \log \mathcal{N}(u; \mu_{\theta}(s), \sigma_{\theta}^2(s)) - \sum_j \log(1 - \tanh^2 u_j)$
stable form	$\log(1 - \tanh^2 u) = 2(\log 2 - u - \text{softplus}(-2u))$
loss	$L_{\pi}(\theta) = \mathbb{E}_{s \sim \mathcal{D}, \xi} [\alpha \log \pi_{\theta}(\tilde{a} \mid s) - \min_i Q_{\phi_i}(s, \tilde{a})]$
pathwise gradient ($\tilde{a}$ depends on θ through $\mu_{\theta}, \sigma_{\theta}$)	$\nabla_{\theta} L_{\pi} = \mathbb{E} [\alpha \nabla_{\theta} \log \pi_{\theta}(\tilde{a} \mid s) + (\alpha \nabla_{\tilde{a}} \log \pi_{\theta}(\tilde{a} \mid s) - \nabla_{\tilde{a}} \min_i Q_{\phi_i}(s, \tilde{a})) \nabla_{\theta} \tilde{a}]$
KL view	$L_{\pi} = \alpha \mathbb{E}_s [D_{\text{KL}}(\pi_{\theta}(\cdot \mid s) \parallel e^{Q(s, \cdot) / \alpha} / Z(s))] + \text{const}$

6 • Temperature update

target entropy	$\bar{\mathcal{H}} = -\dim(\mathcal{A})$
loss	$L(\alpha) = \mathbb{E}_{s \sim \mathcal{D}, \tilde{a} \sim \pi_{\theta}} [-\alpha (\log \pi_{\theta}(\tilde{a} \mid s) + \bar{\mathcal{H}})]$
gradient	$\frac{\partial L}{\partial \alpha} = \hat{\mathcal{H}} - \bar{\mathcal{H}}, \quad \hat{\mathcal{H}} = \mathbb{E}[-\log \pi_{\theta}(\tilde{a} \mid s)]$
with log α	$\frac{\partial L}{\partial \log \alpha} = \alpha(\hat{\mathcal{H}} - \bar{\mathcal{H}})$
log π is detached here: this loss only moves α . Entropy above target lowers α , below target raises it.	

7 • Target critics

Polyak update	$\bar{\phi}_i \leftarrow \tau \phi_i + (1 - \tau) \bar{\phi}_i$ after every gradient step
lag	a jump in ϕ is $1 - (1 - \tau)^k$ absorbed after k steps; time constant $\approx 1/\tau$

8 • Acting, evaluation and action scaling

training	$a = \tanh(\mu_{\theta}(s) + \sigma_{\theta}(s) \odot \xi)$ (uniform random during warm-up)
evaluation	$a = \tanh(\mu_{\theta}(s))$ (deterministic)
env bounds	$a_{\text{env}} = c + h \odot a, \quad c = (\text{high} + \text{low})/2, \quad h = (\text{high} - \text{low})/2$
log-prob in env units	$\log \pi(a_{\text{env}} \mid s) = \log \pi(a \mid s) - \sum_j \log h_j$

9 • Diagnostics worth logging

Q vs. returns	$\mathbb{E}[Q_{\phi_i}(s, a)]$ vs. observed discounted returns (overestimation check)
entropy vs. target	$\hat{\mathcal{H}}$ vs. $\bar{\mathcal{H}}$, and α over time
losses	L_Q (should stay bounded), L_{π} , $L(\alpha)$
Diverging Q values are the classic SAC failure, especially at high UTD. Returns are the final judge.	

10 • Starting values (SAC v2 paper, MuJoCo)

optimizer	Adam, learning rate 3×10^{-4} for $\theta, \phi, \log \alpha$
RL	$\gamma = 0.99, \quad \tau = 0.005, \quad \bar{\mathcal{H}} = -\dim(\mathcal{A})$
data	$B = 256, \quad \mathcal{D} = 10^6, \quad G = 1$ gradient step per env step
networks	2 hidden layers $\times$ 256 units, ReLU
A random warm-up of about 10^4 steps before learning is common in implementations.	

11 • Relatives and variants

SAC v1	adds a separate V network; fixed α (set via reward scale)
TD3	deterministic actor + clipped double-Q + target smoothing
high-UTD critics	REDQ (critic ensembles), DroQ (dropout + layer norm)
discrete SAC	softmax actor; expectations over actions computed exactly